Guardrails, and Making Standards Travel

Lab 2.b — making a written rule refuse to be violated

FSL AI Training · Phase 1 · Week 4

TODAY

Two hours, hands-on

8 min	**Recap + why**	You have documents. Nothing enforces them
5 min	**Warm-up**	The five-minute guardrail everyone leaves with
19 min	**Layers & severity**	Where enforcement lives; block vs nudge
19 min	**Demo — build along**	One guardrail, verified, in YOUR copy
34 min	**Your clause**	Hook → CI gate → reviewer, each checkpointed
10 min	**Review**	False positives, bypasses, residue
8 min	**Hand-off, wrap & retro**	The sixth artifact; reflect; capture

Sums to ~103 — the slack is deliberate, and it gets spent on the verify steps.

RECAP

Where 2.a left us

One sentence: you wrote the law, and there is no police force.

- **You have an ADR.** The store boundary has a recorded reason.
- **You have an NFR you derived.** With per-clause checks — and the unenforceable clauses marked.
- **Both are wired into CLAUDE.md.** Trigger rows + a binding line.
- **And nothing enforces any of it.** That is today.

THE PREMISE

The model layer cannot be the control.

Models are non-deterministic and persuadable. A rule in CLAUDE.md holds most of the time and fails exactly when someone asks for something reasonable-sounding that violates it. Three words for the rest of the block: a GUARDRAIL is any deterministic mechanism — permission rule, hook, CI check — that refuses a violation instead of describing one. A HOOK is an interception point before or after a consequential action. A POLICY is the deterministic rule evaluated against it. Because the check does not depend on which model ran, it survives a model upgrade.

THE MAP

Five layers of enforcement

Each layer catches a different audience. You need fewer than five — but know which.

Layer	Where it lives	Holds against
1 Instruction	CLAUDE.md, docs behind a trigger table	Nothing reliably — it informs
2 Client-side deterministic	.claude/settings.json permissions + hooks	The agent, at authoring time
3 Commit-time	git hooks, commit lint	Anyone committing locally
4 CI gates	workflows, static analysis, custom validators	Anyone merging — agent or human
5 Documented bypass	a written escape hatch with a named approver	Nothing — it keeps 1–4 honest

This repo ships guardrails only at layers 1–2 (its CI runs lint and tests, but no governance gate yet). Today you build in 1–2 and wire ONE layer-4 gate so you feel what it costs. Layers 3 and 5 are taught, not built.

WARM-UP

The five-minute guardrail

Three verdicts, not two: allow (silent), ask (stop and check), deny (never). The middle one is the one teams forget.

.claude/settings.json

```
{
  "permissions": {
    "ask":  ["Bash(rm *)"],
    "deny": [
      "Edit(server/data/seed.json)",
      "Write(server/data/seed.json)",
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(git reset --hard *)",
      "Bash(git clean *)",
      "Bash(npm publish *)",
      "Bash(curl * | bash)"
    ]
  }
}
```

MERGE, never paste: the first two deny entries are ALREADY in your file, shown so you see where yours go — and your hooks block must survive. The handout shows the before/after. Then: ask Claude to force-push, watch it refuse; confirm seed.json is still protected.

LAYER 2

The hook already in your repo

Read this hook before you write yours — particularly its message.

DEFENSE IN DEPTH, THREE WAYS

- A Don't line in CLAUDE.md.
- A permissions.deny entry.
- A PreToolUse hook.
- All three name the same path and the same fix.

.CLAUDE/HOOKS/PROTECT-SEED.JS

```
console.error([
  'BLOCKED: Modifying seed.json
   is not allowed.',
  'Why: it is the canonical reset
   baseline every lab depends on.',
  'Instead: change data via the API
   or db.json, then reset-db.',
  'Done means: seed.json has no
   diff.',
].join('\n'));
process.exit(2);
```

SEVERITY

Block or nudge? Decide, do not default

The event you register on, plus the exit code, IS the severity decision.

Event	What exit 2 does	Use it for
PreToolUse	BLOCKS the call. stderr goes to Claude	Unrecoverable or unambiguous violations
PostToolUse	Cannot block — the tool already ran — but stderr still reaches Claude	Heuristic rules; things you should not interrupt
exit 0	Allows, silently	The 99% case
any other code	Non-blocking; the user sees a hook-error notice	Almost never on purpose

This adds a third tag to 2.a's mechanical/fuzzy: PARTLY — checkable by heuristic, with false positives. That middle case is exactly what nudges are for. Not every guardrail should block: a gate that fires on legitimate work teaches people to route around it.

VERIFICATION

An unverified guardrail is a belief

The procedure, concretely — the 1.b PR on your prep list is the allow case.

BLOCKS THE BAD

- Ask Claude to do the violation on purpose
- Save the refusal text
- That text is your evidence

ALLOWS THE GOOD

- Replay your Lab 1.b change through it
- The hook must stay silent
- This is the half everyone skips

STAYS TRUE

- Re-verify after every narrowing
- Fixing a false positive often reopens the hole
- Both directions, every time

Evidence standard: paste the refusal text into your PR description. That is what "demonstrated in a live agent run" means.

LAYER 5

The bypass you write down

The layer that makes the other four honest.

- **There is always one.** A label, a commit trailer, an APPROVED- comment, a person with merge rights.
- **Undocumented bypasses are the real hole.** Not the missing gate — the escape hatch nobody wrote down.
- **Name the approver.** "APPROVED-X: <reason + who>" beats a silent override every time.
- **Count how often it is used.** A bypass fired weekly is a rule that is wrong, not a team that is lax.

LAYERS 3–5

Taught today, built at scale

A mature corpus we know runs all five — including four custom static-analysis rule files and nine merge-blocking validators.

Layer	What it looks like at scale	Today
3 Commit-time	commit lint, pre-commit checks on staged files	Taught only
4 CI gates	custom static-analysis rules, spec lint, merge-blocking validators	You wire ONE, in your copy
5 Documented bypass	APPROVED- comments with named sign-off, plus telemetry on fire/bypass rates	You document one

Their layer 5 is one comment convention — the smallest thing on the list, and what keeps the other four trustworthy. Governance is not volume.

Rules about AI are just rules.

The same corpus governs its own AI agent with an NFR — numbered MUST clauses, a thresholds table, and per-clause verification methods, covering human confirmation on destructive operations, masking sensitive data before it reaches a model, and structured output validation. There is no separate discipline for governing AI. It is the artifact you wrote last week, pointed at a new subject.

Demo — build along, one guardrail

The read-only database rule, in YOUR copy. Watch the verify step, not the code.

Read-only database unless approved

The same three-layer shape as the seed guard, on a new subject — built along with me.

- **First:** `npm run reset-db` — db.json is generated and git-ignored, so a fresh copy does not have it.
- **The rule.** `db.json` gets changed through the API or a reset — not edited directly.
- **Not the same rule as 2.a's.** The store boundary is about *imports* (`db/store.ts`); this is about *editing a file*. Same family, two rules — today's hook enforces this one.
- **Three layers, again.** A Don't line → a `permissions.deny` entry → a PreToolUse hook that explains itself.

DEMO · THE CHECK

Twenty lines, no dependencies

Node, cross-platform, no packages. Half this room is on Windows.

.claude/hooks/protect-db.js

```
const path = filePath.replaceAll('\\', '/');

if (path.endsWith('server/data/db.json')) {
  console.error([
    'BLOCKED: db.json is generated, not edited.',
    'Why: hand-edits are lost on the next reset and',
    '  are invisible to everyone else.',
    'Instead: change data through the API, or edit',
    '  seed.json via PR and run npm run reset-db.',
    'Done means: your data change survives a reset.',
  ].join('\n'));
  process.exit(2);   // PreToolUse -> blocks
}
process.exit(0);
```

Registered on PreToolUse for Edit|Write — registration JSON is in the handout, and a NEW session is needed after registering: hooks load at session start.

DEMO · VERIFY

Both directions, then the bypass

This is the part of the demo worth your attention.

BLOCKS

- Ask Claude to edit db.json directly
- Refusal message is useful, not just "no"
- Confirm it did NOT touch the file

ALLOWS

- Ask it to edit a file in server/src/repositories/
- No refusal, no noise
- npm test — still green

BYPASS

- Documented in CLAUDE.md
- Instructor approval, via PR
- Not "disable the hook"

If the allow case fails, the guardrail is worse than nothing — it now blocks work, and someone will remove it by Friday.

Checkpoint 1

NOBODY ADVANCES UNTIL THIS IS TRUE

In YOUR copy: Claude is refused when it edits server/data/db.json, and succeeds when it edits a file in server/src/repositories/.

Both directions, out loud. If you only tested the block, you have not finished — run the allow case now.

YOUR STACK

The same pattern, somewhere you will recognise

Illustration, not today's exercise. A rule most teams have written down and almost nobody checks.

Layer	Applied to a stored-procedure convention
1 Instruction	One line: parameters only — never concatenate into executable SQL
2 Client-side deterministic	A hook inspecting the CONTENT of the .sql being written, not just the path
4 CI gate	The same rule module, run again at merge time
5 Documented bypass	APPROVED-DYNAMIC-SQL: <reason + reviewer>
The honest part	The check has real false positives — a static parameterised sp_executesql is fine

Takeaway card: docs/best-practices/enforcing-a-convention.md works the whole shape through — including the clauses no gate can decide. Independent follow-up; office hours supports it.

The hook — enforce what you wrote

Fifteen minutes. The plumbing is given; the policy is yours.

- **Your 2.a clause, revisited.** The residue decisions you record today update that same NFR.
- **Start from the skeleton.** `docs/labs/snippets/hook-skeleton.js` → `.claude/hooks/check-convention.js`; delete the two example rules; put yours in.
- **Register it, then NEW session.** The settings JSON is in the handout — matcher, event, `$CLAUDE_PROJECT_DIR`. Hooks load at session start.
- **Choose the severity on purpose.** PreToolUse blocks; PostToolUse nudges. If your clause is PARTLY, nudge — and write down why.
- **Verify both directions.** Violation on purpose → save the refusal. Your 1.b change → silence. Refusal text goes in the PR.

CHECKPOINT

Checkpoint 2

NOBODY ADVANCES UNTIL THIS IS TRUE

Your hook refuses the violation, stays silent on your Lab 1.b change, and the refusal text is pasted in your PR description.

The refusal text is the evidence standard — "it worked" is not verification.

The same clause at merge time

Ten minutes. This is the one layer-4 gate you build today.

- **The hook stops the agent. This stops anyone.** Including a human who never opened Claude Code.
- **Start from the gate skeleton.** `docs/labs/snippets/governance-gate.yml` → `.github/workflows/governance.yml`; encode the mechanical core only.
- **`fetch-depth: 0` or it breaks.** The default shallow clone makes any diff against the base branch fail with an unhelpful error.
- **Fail or annotate — decide.** What does a docs-only PR do? Path filter, warn, or accept — on purpose, written down.
- **Record the residue.** Every clause the gate cannot hold: accepted risk, human gate, or dropped — with a name, back in the 2.a NFR.

YOUR CLAUSE · 3 OF 3

Point the reviewer at your standards

This is what "fed to the agent as explicit constraints" actually looks like.

FEEDING THE AGENT CONSTRAINTS

- The repo ships a review subagent that reads CLAUDE.md, then whichever ADRs / NFRs / rules apply.
- Add YOUR two docs to its reading list.
- Run it on your own branch.
- Its findings now cite your standard — that is the loop closing.

4 MINUTES

```
# .claude/agents/
#   taskboard-code-reviewer.md

# add to its process list:
- docs/adr/0002-<yours>.md
- docs/nfr/0002-<yours>.md

# no .claude/agents/ folder?
# your copy predates it — pull it:
npx giget gh:mike-tajmajer-
  fullstacklabs/fsl-taskboard-lab/
  .claude/agents .claude/agents
```

REVIEW

Let us compare

Ten minutes. Two or three screens, not everyone.

- Whose guardrail has a false positive — and how did you find out?
- Who chose nudge over block, and what was the reasoning?
- Whose bypass is documented, and who approves it?
- What went in the residue column — and whose name is against it?
- What does your gate do on a docs-only PR — and was that on purpose?

HAND-OFF

The sixth artifact, and it is yours

2.a named five artifacts that live in a repo. The sixth is the only Collective-layer one — how a standard reaches every repo — and we are deliberately not handing you the answer, because choosing the mechanism, the rung, and the owner IS the roll-up.

WHAT THE GUIDE GIVES YOU

- Four words that carry it: standard, drift, pin, enforce.
- CRAWL: one repo + two CLAUDE.md lines — a 4-step checklist, ~30 min.
- WALK: queryable (3 tool options, traps named) + pinned copies.
- RUN: gates, served docs, a versioned plugin — each needs an owner.
- And a fill-in template for recording what you chose.

THE DECISION TEMPLATE

```
# docs/best-practices/
#   distributing-standards.md

# what "done" looks like:
Owner:       <a person, not a team>
Rung:        crawl | walk | run
First move: <the ~30-min step>
Decided:     YYYY-MM-DD
Next review: YYYY-MM-DD

# ...plus: where standards live,
# how a project gets them, current-
# ness, enforced vs advisory,
# bypass, NOT-doing-yet
```

What leaves this room

Questions & support: #extra-duty-solutions-ai-training — your instructor and the Applied AI Engineer are there.

- **Ship checklist.** Warm-up merged (seed rules intact) · hook + CI gate, both verified both ways, refusal text in the PR · severity reasons written · bypass documented · residue owned in the 2.a NFR · reviewer reading your docs · "how we use these" README (async OK, still reviewed) · roll-up decision with an owner · port target named.
- **Reflection note, now.** 3–5 sentences in your PR: what you learned, what surprised you, what you'd apply tomorrow. Acceptance criterion, not homework.
- **One retro question.** What was hard, what was easy — thirty seconds each.
- **Three continuations.** Port one artifact in office hours (bring the repo, not a question) · define how standards travel (owner + template) · apply the pattern to a convention of your own (the takeaway card).